

Report Agro-Bot

Botteghi Matteo, Guerra Nicholas, Qorri Cristian

2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	3
2	Design	4
2.1	Architettura	4
2.2	Design dettagliato	6
2.2.1	Botteghi Matteo	6
2.2.2	Guerra Nicholas	8
2.2.3	Qorri Cristian	12
3	Sviluppo	15
3.1	Testing automatizzato	15
3.2	Note di sviluppo	16
3.2.1	Botteghi Matteo	16
3.2.2	Guerra Nicholas	16
3.2.3	Qorri Cristian	17
4	Commenti finali	18
4.1	Autovalutazione e lavori futuri	18
4.1.1	Botteghi Matteo	18
4.1.2	Guerra Nicholas	18
4.1.3	Qorri Cristian	19
4.2	Difficoltà incontrate e commenti per i docenti	21
A	Guida utente	22

1 Analisi

1.1 Descrizione e requisiti

Il software **Agro-Bot** è un videogioco gestionale con visuale 2D top-down incentrato sulla gestione di una fattoria futuristica. L'esperienza di gioco ruota attorno al controllo di un drone agricolo. L'obiettivo principale è coltivare il terreno e vendere il raccolto al fine di massimizzare i profitti economici. Il giocatore sarà costantemente chiamato a bilanciare la pianificazione delle attività agricole con l'efficienza logistica: ogni spostamento e ogni azione sul campo (come arare, seminare o innaffiare) comporterà un consumo della batteria del drone, rendendo necessari ritorni strategici alla base di ricarica. Inoltre, l'irrigazione delle colture svuoterà progressivamente il serbatoio dell'acqua a bordo del drone, il quale dovrà essere riempito presso un apposito pozzo. Il ciclo economico del gioco si chiude con la vendita delle colture giunte a maturazione; i ricavi ottenuti forniranno i fondi necessari per acquistare ulteriori semi e permettere il proseguimento della partita.

Requisiti funzionali

- Il sistema dovrà gestire il movimento bidimensionale del drone all'interno dell'area di gioco (griglia).
- Il sistema dovrà rappresentare e aggiornare costantemente lo stato del terreno (es. non arato, arato, seminato, innaffiato).
- Le risorse del drone (livello della batteria, quantità di acqua nel serbatoio e crediti) dovranno essere gestiti dal sistema e decrementati/incrementati in base alle azioni intraprese.
- Il software dovrà implementare due tipologie differenti di colture, ciascuna caratterizzata da logiche e tempistiche di crescita indipendenti.
- Il sistema dovrà fornire un'interfaccia utente (HUD - Heads Up Display) per mostrare al giocatore le informazioni vitali in tempo reale (risorse, batteria, acqua) e meteo corrente.
- Sarà presente un sistema di compravendita che permetta l'acquisto di semi e la vendita del raccolto in cambio di crediti.
- Il sistema integra variazioni meteorologiche e la comparsa spontanea di infestanti (es. erbacce) da debellare.
- Il giocatore ha la possibilità di spendere crediti per acquistare potenziamenti per le caratteristiche del drone (es. capacità serbatoio, batteria maggiorata).

Requisiti non funzionali

- **Reattività e fluidità:** La gestione degli input utente per il movimento e le azioni del drone dovrà essere estremamente fluida, garantendo un'esperienza reattiva.
- **Sincronizzazione:** Il sistema dovrà garantire un'adeguata sincronizzazione dei tempi logici per gestire in parallelo e senza incongruenze il movimento del drone e la crescita asincrona delle colture.
- **Portabilità:** Trattandosi di un progetto basato su Java, l'applicazione dovrà essere multi-piattaforma e correttamente eseguibile su qualsiasi sistema desktop provvisto di Java Virtual Machine.

1.2 Modello del Dominio

Il dominio è incentrato sull'ambiente agricolo rappresentato da una griglia (**Grid**) suddivisa in singole celle (**Cell**). Ogni cella possiede uno stato specifico (es. arata, secca, bagnata) e può ospitare una coltura (**Crop**). Le colture hanno un proprio ciclo di vita suddiviso in stadi di crescita.

All'interno del gioco si muove il drone agricolo (**Drone**), che rappresenta il mezzo d'interazione principale. Il drone è equipaggiato con una batteria (**Battery**) per la riserva energetica e un serbatoio (**WaterTank**) per l'irrigazione. Il gioco include anche specifiche strutture indispensabili per il ciclo di gioco: una base di ricarica (**Storage**) per ripristinare l'energia del drone e conservare oggetti in un inventario a parte, un pozzo (**Well**) da cui attingere l'acqua e un mercato (**Market**) presso il quale comprare semi e vendere i raccolti per generare ricavi.

Il giocatore (**Player**) impartisce i comandi al drone e possiede un portafoglio di crediti (**Wallet**) necessario per le transazioni economiche. La vera difficoltà del problema (non legata all'implementazione) è gestire entità che cambiano nel tempo in modo indipendente (in particolare la progressione dello sviluppo vegetativo delle colture) in simultanea rispetto alle azioni manuali e immediate del drone.

Gli elementi costitutivi del problema sono sintetizzati nel diagramma UML sottostante (Figura 1).

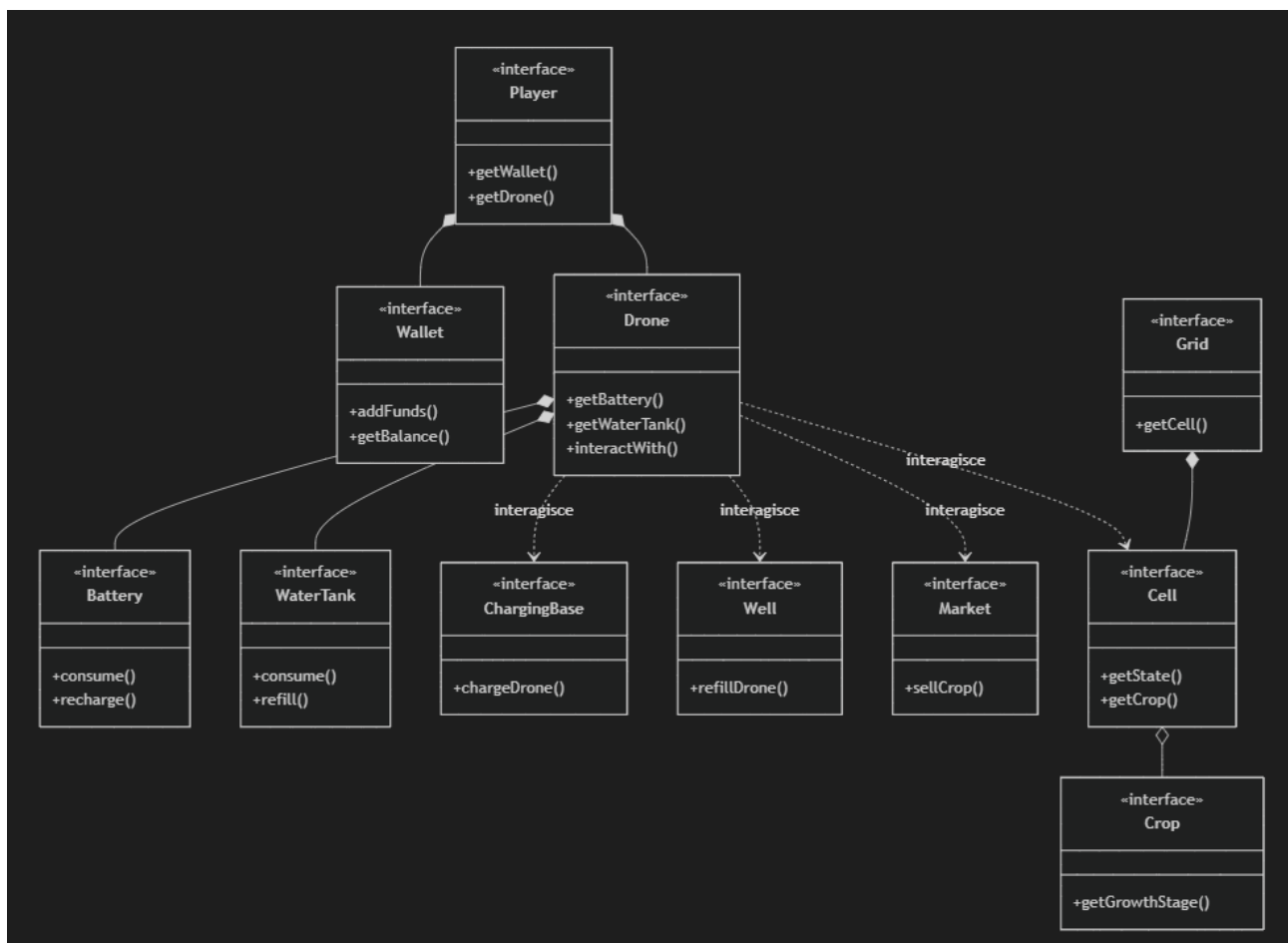


Figura 1: Elementi costitutivi del dominio applicativo di Agro-Bot.

2 Design

2.1 Architettura

L'architettura di Agro-Bot si basa sul pattern Model-View-Controller (MVC), specificamente declinato per supportare le dinamiche interattive di un videogioco in tempo reale tramite l'ausilio aggiuntivo di un Game Loop.

Questa scelta permette di separare nettamente i dati e la logica di dominio dalla loro rappresentazione grafica e dalla cattura degli input dell'utente.

- Model:**
- Racchiude lo stato dell'applicazione e la logica di dominio del contesto agricolo.
 - I principali punti di accesso sono rappresentati dalle entità **Drone** (implementato da **DroneImpl**), **Grid** (implementata da **GridImpl**) e dai gestori di servizio come **WeatherManager** e **Market**.
 - È completamente disaccoppiato da Vista e Controller: non contiene alcun riferimento a librerie grafiche o sistemi di input.
 - La sua responsabilità primaria è garantire l'integrità e la consistenza dello stato interno.
- View:**
- Si occupa esclusivamente del rendering visivo dello stato del gioco a schermo.
 - I punti di ingresso principali sono **GameWindow** (il frame principale) e **GamePanel** (il canvas principale), che delega il rendering di dettaglio a viste specializzate quali **GridView**, **DroneView** e **HUDView**.
 - Interroga passivamente il model per estrarre le informazioni necessarie da disegnare ad ogni frame.
- Controller:**
- Coordina l'interazione tra View e Model gestendo il flusso applicativo attraverso tre componenti chiave:
 - **InputHandler**: intercetta gli eventi generati dall'utente (pressione dei tasti) e li traduce in comandi che alterano lo stato del model (es. movimento del drone, lavorazione delle celle).
 - **GameLoop**: thread dedicato con frequenza costante; ad ogni ciclo forza l'avanzamento del tempo logico del modello (es. aggiornamento del meteo) e notifica la vista (**GamePanel**) invocando il metodo **repaint()**.
 - **GameStateManager**: funge da router per coordinare i cambi di contesto della schermata (menu principale, partita attiva, pausa, game over).

In Figura 2 è schematizzato il diagramma UML architetturale di Agro-Bot, che evidenzia i pattern di comunicazione tra le componenti MVC.

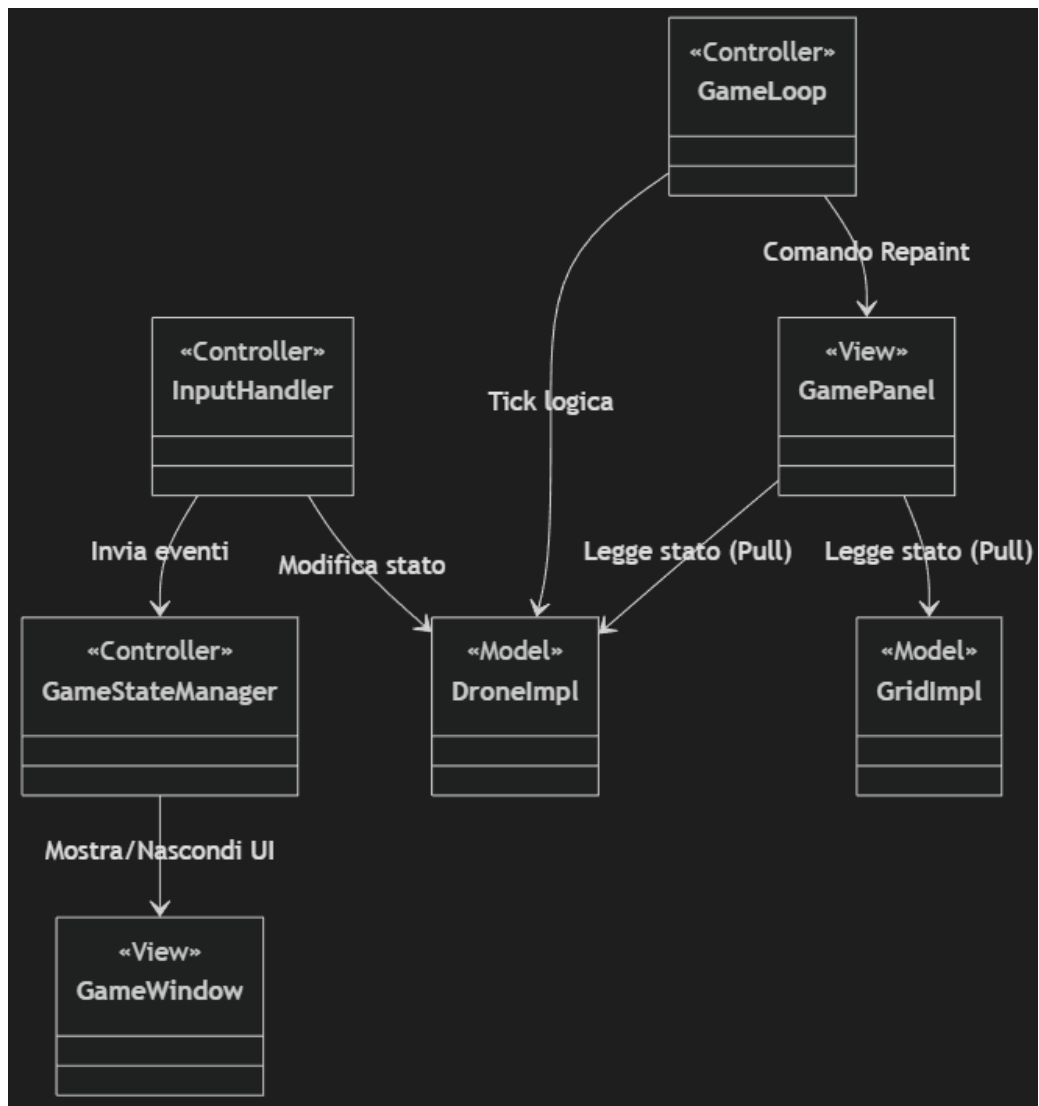


Figura 2: Diagramma UML architetturale di Agro-Bot: pattern MVC e comunicazione tra componenti.

2.2 Design dettagliato

2.2.1 Botteghi Matteo

Entità Drone e Gestione Risorse (Pattern Facade)

Problema: Analizzando i requisiti, era chiaro che il drone dovesse fare parecchie cose: sapere le proprie coordinate, gestire la batteria, controllare quanta acqua c'è nel serbatoio, gestire l'inventario degli oggetti raccolti e anche i soldi (il wallet). Se fossero stati inseriti tutti i controlli e il codice all'interno di un'unica classe **Drone**, ne sarebbe uscita una "God Class". Questo avrebbe violato il principio di Singola Responsabilità (SRP), rendendo il codice difficile da leggere e complicato da debuggare.

Soluzione: Per evitare questo problema, si è optato per la composizione anziché per ereditarietà. Sono quindi state create classi piccole e indipendenti come **Battery**, **WaterTank**, **Inventory** e **Wallet** per suddividere le diverse responsabilità. L'implementazione del drone (**DroneImpl**) semplicemente contiene le istanze di queste classi e le usa. Ho applicato il pattern **Facade**: l'interfaccia **Drone** fa da facciata e nasconde la complessità di quello che c'è dietro. Quando classi esterne chiamano un'azione (ad esempio `irrigate()`), ci pensa internamente **DroneImpl** a scalare le risorse dai vari moduli, mantenendo il sistema molto più pulito e flessibile rispetto a una rigida gerarchia di ereditarietà.

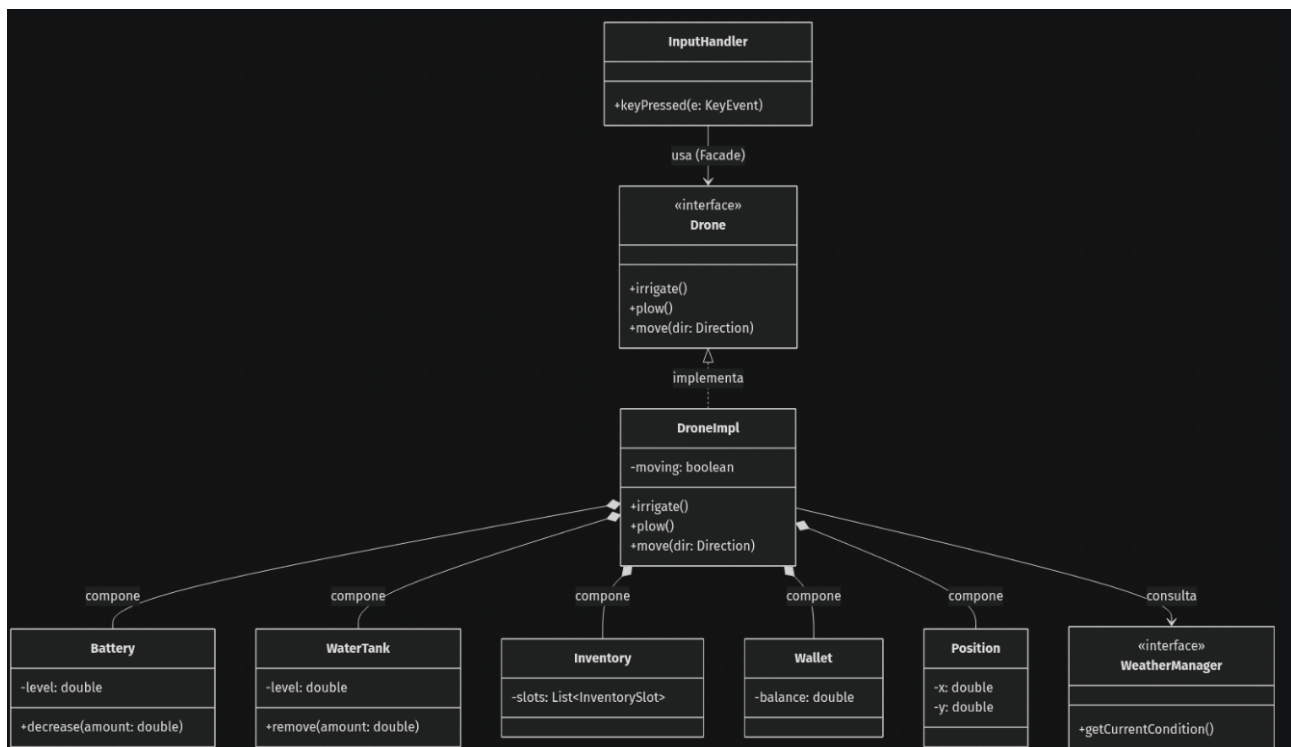


Figura 3: Diagramma delle classi del modello Drone. L'interfaccia **Drone** agisce da "Facciata" (Pattern Facade) per i componenti esterni, come l'**InputHandler**. In questo modo viene nascosta tutta la complessità della classe **DroneImpl**, che al suo interno unisce e gestisce i moduli per le risorse (**Battery**, **WaterTank**, ecc.), la posizione e il meteo.

Gestione del Movimento Fluido e Allineamento alla Griglia (Pattern Update Method)

Problema: Il gioco si basa su una griglia fissa divisa in caselle, ma un obiettivo che ci siamo posti era avere un movimento fluido sullo schermo, senza "teletrasporti" a scatti da una casella all'altra. Il problema di design era come calcolare questa traslazione continua senza incastrarsi con eventuali cali di frame rate del computer e garantire che alla fine del movimento, il drone risultasse allineato in modo perfetto alla casella di arrivo.

Soluzione: Si è optato per separare il momento in cui si preme il tasto dal momento in cui il drone si sposta fisicamente, scartando l'idea di usare cicli bloccanti (come `Thread.sleep()`) che avrebbero congelato l'intero Game Loop. Quando si dà l'input, il metodo salva solo la destinazione. Per risolvere il problema è stato utilizzato l'**Update Method**: il drone espone un metodo `updateState(double deltaTime)` che viene chiamato in continuazione dal Game Loop principale. Il drone usa il tempo trascorso (`deltaTime`) per calcolare di quanto spostarsi in modo incrementale e slegato dagli FPS. Raggiunta la destinazione, l'animazione si ferma e le coordinate vengono forzate ai valori esatti e interi della griglia logica.

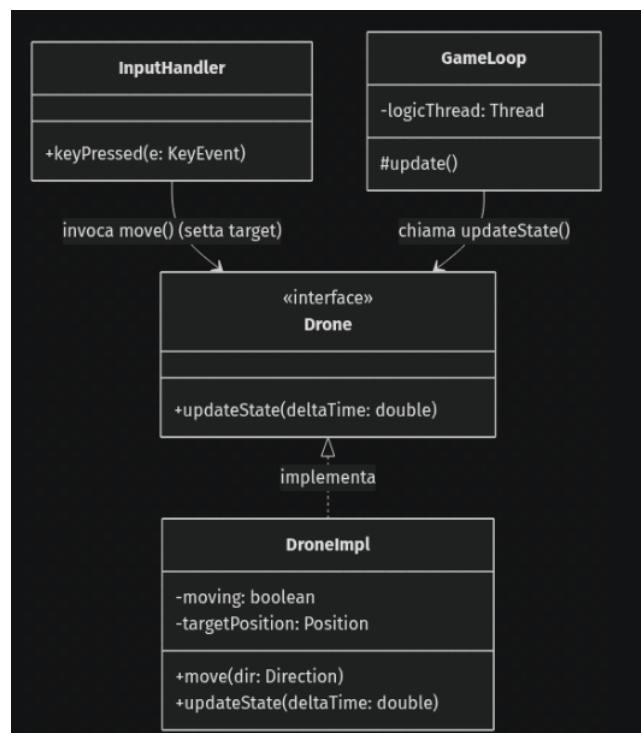


Figura 4: Diagramma che mostra il pattern Update Method usato per il movimento. È evidente la separazione tra il comando (dato dall'**InputHandler**) e lo spostamento fisico sullo schermo. È il **GameLoop** a chiamare continuamente il metodo `updateState(deltaTime)`, calcolando la posizione in base al tempo trascorso e garantendo un movimento fluido

2.2.2 Guerra Nicholas

Gestione dei cambi di stato (Pattern Observer)

Problema: L'applicazione deve gestire numerosi stati di gioco (Menu Principale, Gioco Attivo, Pausa, Menu Negozio, Menu Magazzino, Game Over). Al variare dello stato, l'interfaccia grafica deve reagire dinamicamente, nascondendo o mostrando i pannelli appropriati (es. `GamePanel`, `PauseMenuUI`, `MainMenuView`). Gestire questi cambi direttamente all'interno delle classi di logica o interfacciare esplicitamente ogni singolo componente della UI agli altri, avrebbe creato una forte interdipendenza ("spaghetti code"), aumentando notevolmente il rischio di errori nell'aggiunta di nuove schermate.

Soluzione: Si è optato per l'utilizzo del pattern **Observer**, semplificato e orientato alla programmazione funzionale tramite l'uso di `Consumer` in Java. È stata introdotta la classe `GameStateManager` che funge da *Observable*. Questa classe gestisce lo stato del gioco (`GameState`) e permette di registrare un listener tramite `setOnStateChange()`. La classe `App` agisce da *Observer*: registra una funzione di callback che, alla notifica di un cambiamento di stato, si occupa di invocare i metodi corretti sulla `GameWindow` per effettuare lo switch dei pannelli visivi. In questo modo, i componenti reattivi come l'`InputHandler` possono richiedere un cambio di stato (es. `stateManager.setState(PAUSED)` alla pressione del tasto ESC) senza avere alcuna conoscenza di come la GUI gestirà effettivamente la transizione a schermo.

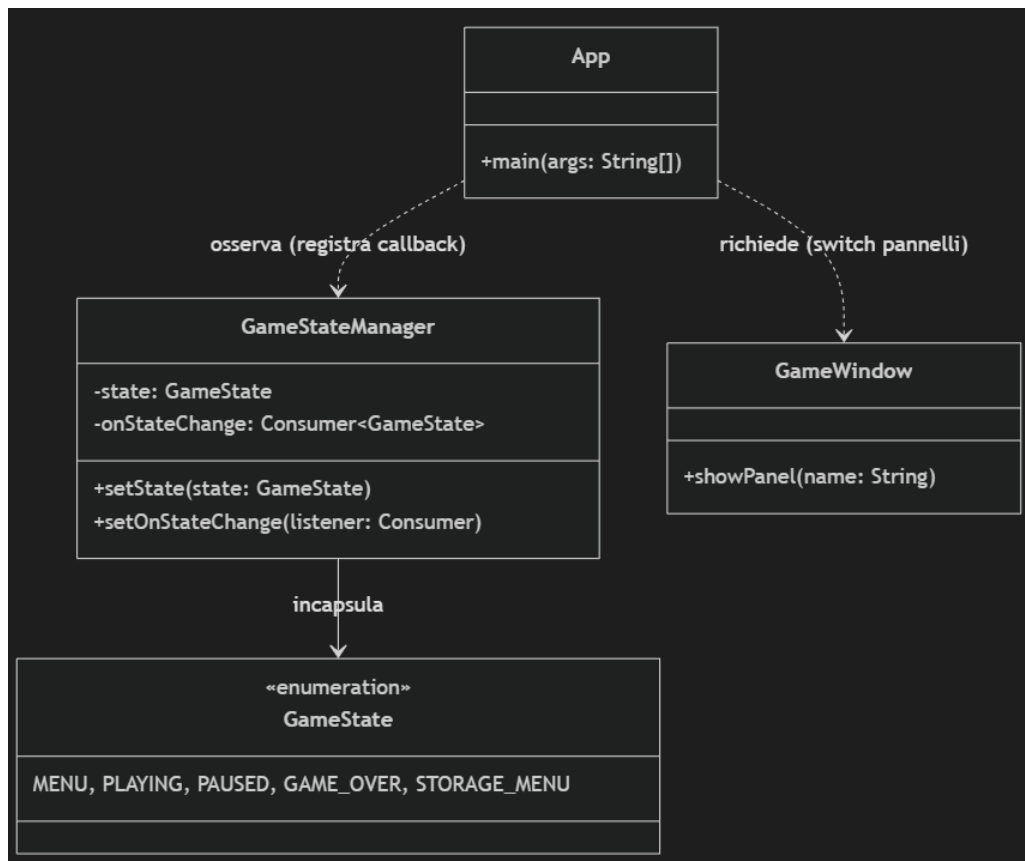


Figura 5: Diagramma del pattern Observer applicato alla gestione degli stati di gioco in Agro-Bot.

Sincronizzazione Logica e Rendering (Pattern Game Loop)

Problema: In un videogioco gestionale in tempo reale, il mondo di gioco deve evolversi in maniera indipendente dagli input dell'utente. Le colture devono crescere, il meteo deve cambiare e i parametri operativi del drone devono essere aggiornati progressivamente. Contemporaneamente, l'interfaccia grafica deve aggiornarsi in modo fluido, senza che pesanti calcoli logici rallentino il rendering e senza che ritardi nel disegno interferiscano con i tempi di esecuzione della logica.

Soluzione: È stato implementato il pattern architetturale **Game Loop**, considerato lo standard nello sviluppo software di applicazioni real-time. La classe **GameLoop** implementa l'interfaccia **Runnable** e gestisce internamente due **Thread** separati:

- Il **Logic Thread** (che cicla sul metodo `update()`) dedicato all'avanzamento dei tempi fisici del modello (UPS - Updates Per Second).
- Il **Render Thread** (che cicla sul metodo `renderLoop()`) incaricato di forzare il ridisegno della vista grafica (FPS - Frames Per Second).

Per evitare problemi di *race condition* (ad esempio: il Render Thread tenta di leggere e disegnare lo stato di una cella nel momento esatto in cui il Logic Thread la sta modificando e distruggendo), è stato impiegato un oggetto di sincronizzazione (`stateLock`) per garantire la mutua esclusione. I blocchi `synchronized(stateLock)` all'interno dei due cicli garantiscono un accesso sicuro allo stato dell'applicazione, disaccoppiando le frequenze di aggiornamento logico da quelle grafiche.

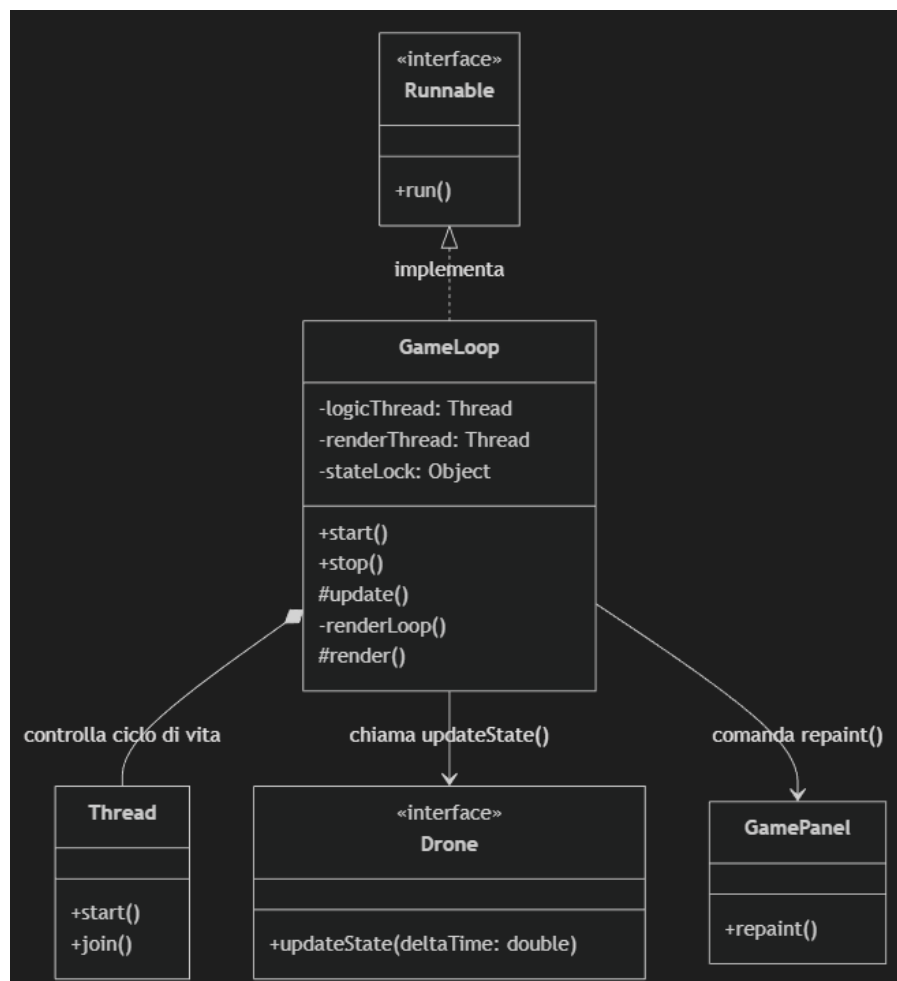


Figura 6: Diagramma del pattern GameLoop: separazione tra Logic Thread e Render Thread con sincronizzazione tramite `stateLock`.

Semplificazione dei sottosistemi (Pattern Facade)

Problema: All'interno del negozio (**ShopMenuUI**), l'utente può vendere i raccolti o acquistare semi e potenziamenti. Il completamento di una transazione richiede l'interazione coordinata di molteplici sottosistemi logici: bisogna verificare se si possiede fisicamente l'oggetto nell'inventario (**Inventory**), interrogare il listino prezzi aggiornato (**PriceManager**) per sapere quanto vale e infine accreditare o scalare la somma dal portafoglio del giocatore (**Wallet**). Far comunicare la classe che gestisce l'interfaccia grafica direttamente con tutte queste entità avrebbe creato un elevato grado di interdipendenza.

Soluzione: È stata introdotta la classe **Market** che funge da *Facade* (Facciata strutturale). Il **Market** racchiude e nasconde del tutto la complessità delle interazioni tra **Inventory**, **Wallet** e **PriceManager**, esponendo all'esterno (alla UI) esclusivamente metodi semplici e ad alto livello come **sellCrop(nome)** o **buyBatteryUpgrade(drone)**. Grazie a questo design pattern, le classi visive (come **ShopMenuUI**) dialogano con il dominio utilizzando solo la facciata **Market**, astruendo dalle complesse logiche di validazione economica e inventariale sottostanti. Ciò garantisce un codice più manutenibile e un modello solido.

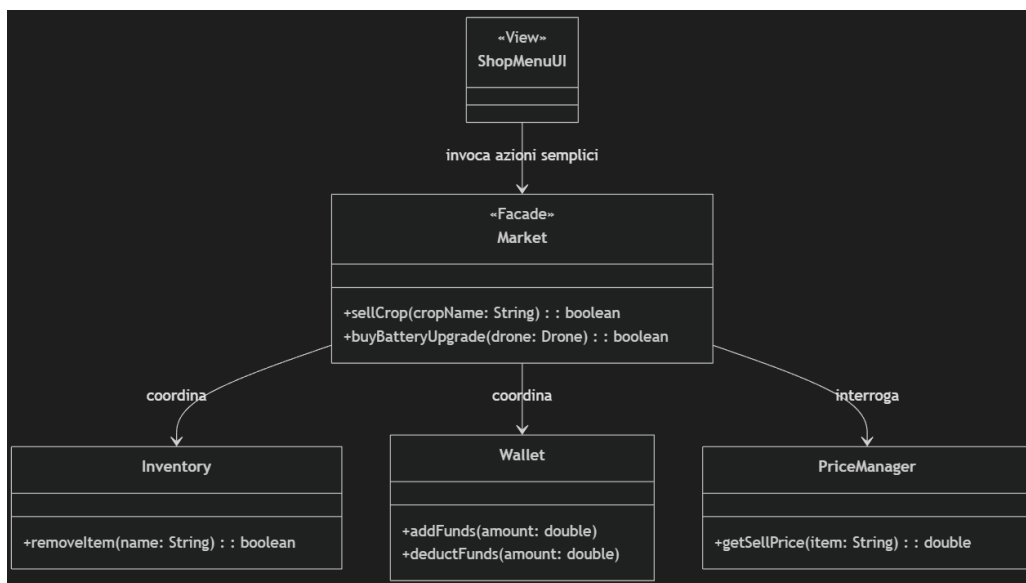


Figura 7: Diagramma del pattern Facade: la classe **Market** semplifica l'interazione tra UI e sottosistemi di inventario, prezzi e portafoglio.

Selezione attiva dell'inventario (State/Iterator Pattern)

Problema: Durante la fase di semina, il giocatore deve poter scegliere quale tipo di seme piantare (es. grano o mais) tra quelli disponibili nel proprio inventario. Implementare questa meccanica tramite interfacce grafiche complesse (come come menu in sovrapposizione o drag-and-drop col mouse) avrebbe interrotto bruscamente il ritmo di gioco real-time basato sulla tastiera. D'altro canto, delegare la responsabilità della scelta all'**InputHandler** (costringendolo a memorizzare quale seme l'utente volesse piantare) avrebbe reso il controller eccessivamente dipendente dallo stato dell'inventario.

Soluzione: L'entità **Inventory** memorizza lo slot correntemente selezionato (**selectedSlotIndex**), comportandosi di fatto come un *Active Cursor* (che unisce i concetti del pattern *State* e dell'*Iterator*). La logica di selezione è racchiusa all'interno dell'inventario. Tramite la pressione dei tasti numerici (1, 2, 3), l'**InputHandler** si limita a comandare lo spostamento del "cursore" interno all'inventario (tramite il metodo **setSelectedSlotIndex()**). Quando il giocatore preme il tasto 'P' per piantare, il controllore interroga semplicemente l'inventario per farsi restituire lo slot correntemente attivo (**getSlot(selectedIndex)**), estraendone il seme. Simultaneamente, la **HUDView** legge passivamente questo stato in maniera del tutto disaccoppiata per evidenziare visivamente (con un riquadro giallo) lo slot selezionato a schermo.

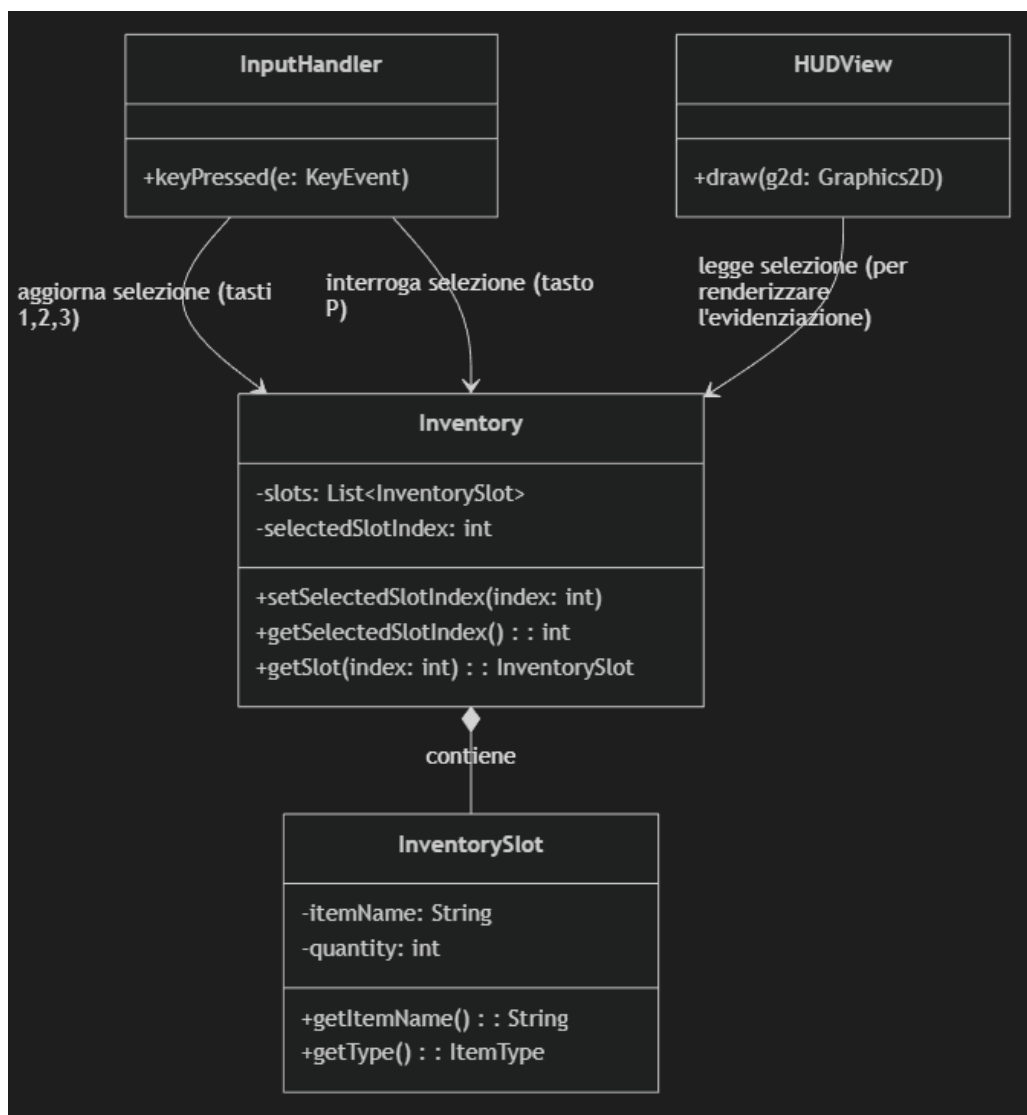


Figura 8: Diagramma del pattern State/Iterator: **Inventory** come cursore attivo per la selezione degli slot.

2.2.3 Qorri Cristian

Astrazione delle colture e riuso del codice

Problema: All'interno del dominio del gioco, una singola zolla di terra (rappresentata dalla classe `TileImpl`) deve essere in grado di ospitare molteplici tipologie di piante (come grano, mais, ecc.). Ogni specie vegetale è caratterizzata da parametri unici, quali il tempo necessario per il completamento della crescita e il livello di tolleranza alla siccità. La sfida progettuale principale consisteva nel permettere alla zolla di gestire l'intero ciclo di vita della pianta (avanzamento della crescita, stato di idratazione e raccolto) mantenendo un totale disaccoppiamento. Era fondamentale evitare che la zolla dovesse conoscere a priori i dettagli specifici del tipo di coltura piantata. Un approccio ingenuo avrebbe infatti richiesto l'implementazione di una complessa logica condizionale (es. una cascata di costrutti `if/else` o `switch`) all'interno della zolla per differenziare le operazioni in base alla pianta. Questo avrebbe generato un'elevata duplicazione di codice e reso il sistema fragile e difficilmente estensibile (violando il principio `Open/Closed`).

Soluzione: Per risolvere tale criticità, l'architettura adottata si fonda sulla programmazione alle interfacce e sull'utilizzo di una gerarchia polimorfica. È stata introdotta l'interfaccia `Crop`, che unisce la zolla e una qualsiasi coltura. Grazie al polimorfismo, `TileImpl` interagisce esclusivamente con l'astrazione `Crop` tramite metodi generici quali `water()` e `update()`, ignorando completamente l'effettiva implementazione sottostante.

Inoltre, con l'obiettivo di massimizzare il riuso del codice e centralizzare i comportamenti comuni, è stata implementata la classe astratta `AbstractCrop`. Questa superclasse fornisce l'implementazione concreta di tutta la logica di base condivisa dalle piante: l'avanzamento dei timer interni per la crescita, l'idratazione e la gestione automatica delle transizioni di stato (da seme a pianta matura). Grazie a questa scelta strutturale, le classi concrete (come `Corn` e `Wheat`) risultano estremamente "leggere". Il loro compito si limita all'estensione di `AbstractCrop` e all'invocazione del costruttore della superclasse a cui passano, come argomenti, i propri parametri specifici (come il tempo totale di crescita e il limite massimo di idratazione), delegando tutta la logica operativa all'astrazione superiore.

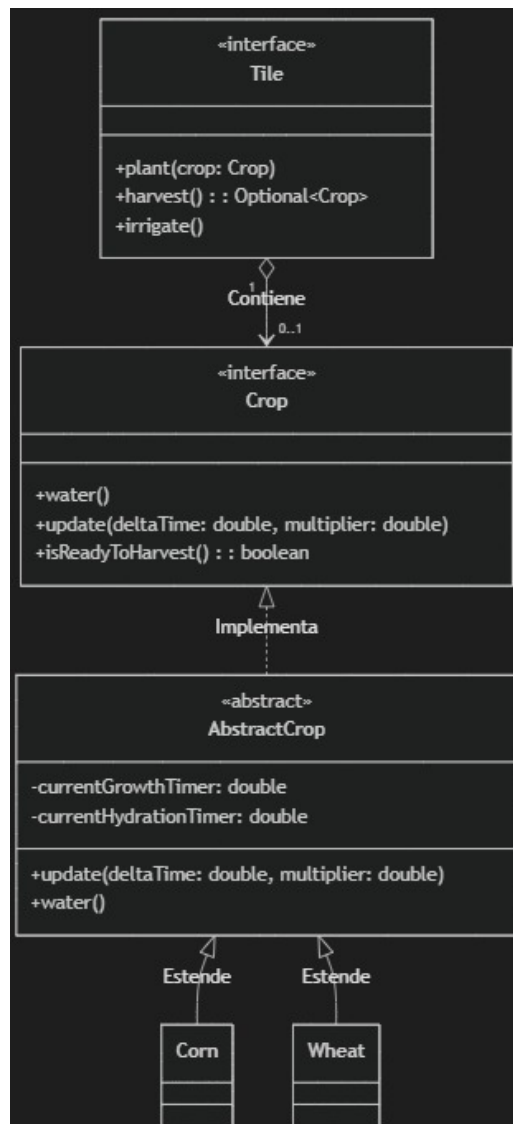


Figura 9: Diagramma UML della gerarchia **Crop**: interfaccia, classe astratta **AbstractCrop** e implementazioni concrete **Corn** e **Wheat**.

Sistema Meteorologico (Update Method Pattern)

Problema: L'implementazione di uno dei requisiti opzionali prevedeva l'introduzione di un sistema meteorologico dinamico, capace di generare precipitazioni casuali in grado di irrigare automaticamente il terreno di gioco. La sfida principale consisteva nel far evolvere lo stato atmosferico in parallelo e in perfetta sincronia con le restanti meccaniche del gioco. Era fondamentale evitare soluzioni computazionalmente "bloccanti" (che avrebbero arrestato o rallentato l'esecuzione del programma durante i calcoli per la pioggia) e allo stesso tempo, prevenire un forte accoppiamento strutturale. Il sistema meteorologico doveva infatti operare come un'entità indipendente, la cui logica temporale non fosse mescolata o rigidamente ancorata a quella del terreno.

Soluzione: Per soddisfare questi requisiti si è adottato un approccio architetturale basato sull'*Update Method Pattern*. Nello specifico, la classe `WeatherManagerImpl` non fa uso di propri thread o cicli infiniti, ma espone un metodo `update(double deltaTime)`. Tale metodo viene invocato ciclicamente ad ogni frame dal `GameLoop` principale, ricevendo in input il tempo reale trascorso dall'ultimo aggiornamento. Questo meccanismo permette al gestore meteo di accumulare internamente il tempo e far evolvere il proprio stato in totale autonomia, mantenendo fluida l'esecuzione del gioco.

I vari stati sono stati modellati in attraverso l'enumerazione `WeatherCondition` (rappresentante, ad esempio, le transizioni da soleggiato a piovoso), garantendo maggiore robustezza al codice.

Infine, per assicurare un solido disaccoppiamento logico tra i componenti, è stato applicato il principio della *Dependency Injection*. L'istanza della mappa di gioco (`Grid`) non viene creata o recuperata globalmente dal gestore meteo, ma gli viene iniettata dall'esterno tramite costruttore. Durante uno stato di pioggia, il `WeatherManagerImpl` seleziona casualmente delle zolle iterando sulla `Grid` ricevuta e si limita a invocare su di esse il metodo `irrigate()`. In questo modo, il gestore del meteo comanda l'azione, ma lascia che sia il terreno ad aggiornare i propri dati (come il timer di idratazione). Questo rende i due componenti indipendenti e ben organizzati.

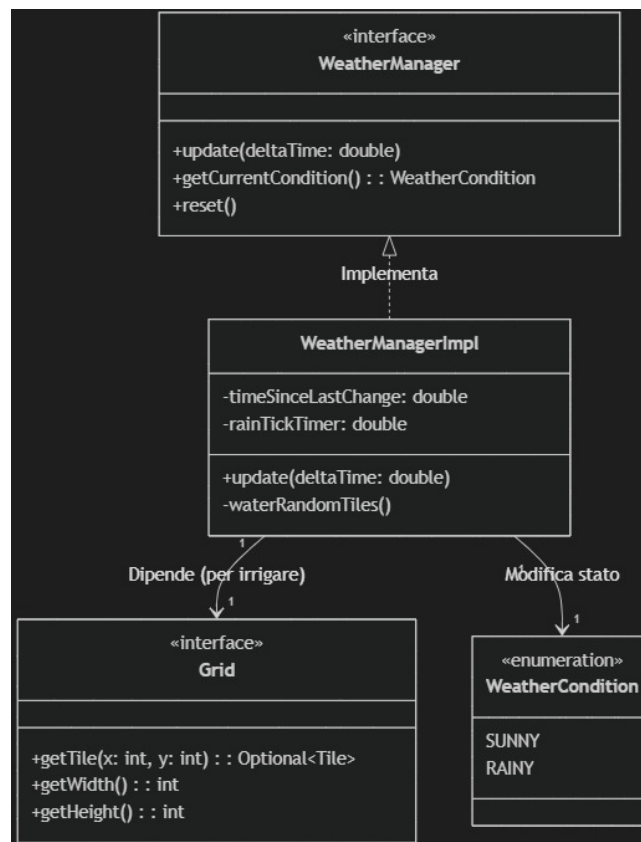


Figura 10: Diagramma UML del sistema meteorologico: `WeatherManagerImpl` con il metodo `update()` e interazione con `Grid` e `WeatherCondition`.

3 Sviluppo

3.1 Testing automatizzato

Per garantire l'affidabilità e la manutenibilità del progetto nel tempo, si è deciso di implementare fin da subito numerosi test automatizzati. Abbiamo utilizzato il framework `JUnit 5`, grazie al quale è stato possibile collaudare le dinamiche principali dell'applicazione e assicurarsi che le nuove implementazioni non introducessero errori su funzionalità già stabili. Tutto ciò è stato realizzato in modo completamente indipendente dall'interazione umana: i test vengono infatti eseguiti in totale autonomia dal sistema di build, senza mai richiedere input manuali o supervisione visiva.

Le aree del software sottoposte a verifica automatica sono state suddivise in base ai principali sottosistemi architetturali:

- **Il dominio logico e le dinamiche di gioco (model):** Sono stati creati test mirati a validare la corretta gestione della mappa e il comportamento del drone in risposta alle istruzioni (tramite classi di test come `GridTest`, `TileTest`, `PositionTest`, `CropTest` e `DroneTest`). In contemporanea, è stata verificata tutta la logistica legata alla capienza e al trasporto delle risorse del drone (`InventoryTest`, `StorageTest`, `BatteryTest`, `WaterTankTest`).
- **Economia e transazioni nel market:** Un aspetto centrale del progetto è rappresentato dalla compravendita di semi e raccolti. Trattandosi di un sistema complesso che coordina portafoglio, listino prezzi e inventario, si è deciso di testare a fondo la robustezza di tutte le varie transazioni. Tramite specifiche classi di test (`MarketBuyTest`, `MarketSellTest`, `PriceManagerTest`, `WalletTest`) ci si è assicurati che il bilancio del giocatore scali/aumenti correttamente, che i prezzi varino come previsto e che sia negata ogni transazione non supportata da sufficiente spazio in inventario o fondi monetari.
- **La gestione degli stati e il ciclo di gioco (controller):** L'engine alla base del progetto deve gestire le tempistiche fisiche del gioco in modo coerente. Per validare tutto ciò, il controller è stato testato su due fronti: da un lato si è verificato il rigoroso tempismo imposto al tick-rate del motore (`GameLoopTest`), dall'altro si è collaudata l'affidabilità con cui l'observable gestisce il passaggio tra i vari menù e lo stato di "gioco attivo" (`GameStateTest`).
- **La gestione della concorrenza e la thread-safety:** Dal momento in cui l'architettura scelta per il game loop prevede che l'aggiornamento della logica e il rendering grafico avvengano su thread separati che lavorano in parallelo, si è presentata la problematica di assicurare un accesso ai dati condivisi che fosse totalmente thread-safe. Per evitare i problemi derivanti dalle race conditions, si è pensato di progettare test dedicati (`ConcurrencyTest`, `HUDViewConcurrencyTest`) forzando l'accesso simultaneo alle risorse, validando il corretto funzionamento dei blocchi `synchronized` in tutto il dominio di gioco.

Questo approccio ha consentito di affrontare l'integrazione delle diverse componenti con grande sicurezza, potendo contare su un feedback sicuro, rapido e del tutto automatizzato a ogni singola modifica del codice sorgente.

3.2 Note di sviluppo

3.2.1 Botteghi Matteo

- **Uso di Stream, di Optional o di altri costrutti funzionali:** Ho utilizzato `Optional` per gestire la ricerca di una casella della mappa. Poiché il drone potrebbe trovarsi fuori limite e non trovare nulla, l'uso di `Optional` combinato al metodo `ifPresent()` mi ha permesso di evitare i classici e fastidiosi controlli `null`, rendendo il codice più pulito.

[Permalink](#)

- **Uso di lambda expression:** Ho sfruttato le lambda expressions come istruzioni rapide da passare proprio al metodo `ifPresent()`. In questo modo, il codice che applica il diserbante viene eseguito solo se la casella esiste, senza dover scrivere intere classi separate per gestire l'azione.

[Permalink](#)

- **Uso di parti della libreria JDK non spiegate a lezione (ImageIO):** Per poter disegnare il drone sullo schermo, avevo bisogno di leggere la sua immagine dai file di gioco. Ho studiato e utilizzato le classi avanzate `ClassLoader` e `javax.imageio.ImageIO`, che permettono di caricare e decodificare dinamicamente le immagini direttamente dalle cartelle del progetto.

[Permalink](#)

- **Uso di librerie di terze parti:** Ho utilizzato la libreria esterna JUnit Jupiter per poter sviluppare e lanciare in automatico l'intera suite di test del drone, sfruttando i suoi strumenti integrati per verificare la correttezza dei risultati.

[Permalink](#)

3.2.2 Guerra Nicholas

- **Uso di stream e collectors per la gestione dei dati:** Per implementare le logiche commerciali del market (`PriceManager`), mi sono trovato a dover gestire numerosi dati in Mappe. Invece di istanziare collezioni temporanee e ciclarle manualmente con `for-each`, ho pensato di utilizzare gli stream (`buyPrices.entrySet().stream()`). Questo mi ha permesso di creare pipeline funzionali per filtrare, mappare e ricompilare i risultati in nuove strutture dati (`Collectors.toSet()`) in modo preciso.

[Permalink](#)

- **Uso di lambda expressions:** Ho fatto uso delle espressioni lambda lungo tutto il progetto. Per gestire i cambi schermata in modo totalmente distaccato, ho utilizzato logiche di callback passando espressioni lambda all'interno del `GameStateManager`. In modo simile, ho spostato i comportamenti personalizzati dall'interfaccia grafica alla logica del programma, passando dei blocchi di codice (`Runnable`) all'`InputHandler`. Usare le funzioni come parametri ha reso i componenti molto più indipendenti e mi ha permesso di avere un codice più pulito, eliminando numerose righe lunghe e ripetitive.

[Permalink](#)

- **Gestione avanzata della concorrenza (Multi-threading e Monitor Objects):** Considerata la natura dinamica e real-time del videogioco, ho dovuto implementare un'infrastruttura (il `GameLoop`) capace di gestire l'avanzamento temporale (`update()`) e il rendering grafico (`render()`) su due Thread paralleli. Lavorare con la programmazione concorrente porta spesso al problema delle race conditions, che si verificano quando l'interfaccia e la logica cercano di modificare le stesse variabili contemporaneamente. Per proteggere i dati senza rallentare il programma con scatti (stuttering), ho evitato di sincronizzare interi metodi. Ho pensato invece di usare blocchi `synchronized` mirati su specifici oggetti di blocco (`stateLock`), proteggendo solo le parti di codice strettamente necessarie.

[Permalink](#)

- **Uso di enhanced switch expressions:** Per gestire gli input da tastiera e il passaggio tra le varie schermate (stati grafici), ho sostituito il costrutto `switch-case` con le `switch expressions` di Java. Usando l'operatore freccia (`->`) si evitano gli errori accidentali di esecuzione a cascata (fall-through) e non c'è più bisogno di ripetere continui `break`. Inoltre, questo approccio mi ha permesso di raggruppare più tasti sulla stessa riga rendendo il codice molto più pulito e facile da leggere.

[Permalink](#)

- **Integrazione nativa OS per feedback uditivo:** Per migliorare la User Experience, volevo fornire al giocatore un feedback immediato qualora tentasse un'azione invalida (come raccogliere da una cella vuota). Piuttosto che riempire l'interfaccia con popup di errore a schermo, ho deciso di utilizzare componenti della libreria JDK (`java.awt.Toolkit`). Invocando `Toolkit.getDefaultToolkit().beep()`, l'applicazione comunica direttamente col sistema operativo dell'utente per emettere l'avviso sonoro nativo, fornendo un feedback uditivo immediato.

[Permalink](#)

3.2.3 Qorri Cristian

- **Utilizzo combinato di optional e approccio funzionale (metodo `ifPresent`):** Durante lo sviluppo dei vari sottosistemi (in particolare il meteo e il controller principale del drone), ho cercato di adottare un paradigma di programmazione sicuro. Quando l'`InputHandler` deve calcolare l'interazione tra il drone e la griglia di gioco, c'è sempre il rischio che le coordinate restituiscano un valore nullo. Invece di riempire il codice di controlli `if (tile != null)`, ho sfruttato l'oggetto `Optional<Tile>`. Concatenando la chiamata con `.ifPresent(...)` e passandovi una lambda expression, il blocco di codice relativo alla ricarica del drone o al rifornimento d'acqua viene eseguito solo se l'oggetto è presente, azzerando il rischio di `NullPointerException` e rendendo il sistema più sicuro.

[Permalink](#)

- **Key bindings:** Per quanto riguarda la GUI, ho riscontrato un problema tipico di Swing: l'uso del `KeyListener` generava difetti legati alla perdita di "focus", impedendo la corretta chiusura dei menu. Per superare questo ostacolo, ho implementato il sistema dei Key Bindings (`InputMap` e `ActionMap`). Questa feature disaccoppia l'evento di pressione del tasto rispetto all'azione logica che ne consegue, garantendo affidabilità anche quando l'utente interagisce col mouse su altri elementi a schermo.

[Permalink](#)

- **Utilizzo di lambda expressions:** Ho sfruttato ampiamente le lambda expressions all'interno delle interfacce. Assegnando un comportamento ai pulsanti dell'inventario (come il trasferimento di un oggetto), le lambda mi hanno permesso di evitare la ripetizione delle classi anonime (`new ActionListener() { ... }`), spostando il focus unicamente sulla logica di trasferimento.

[Permalink](#)

4 Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Botteghi Matteo

Il mio ruolo nel gruppo Il mio ruolo all'interno del team si è concentrato sulla progettazione e lo sviluppo centrale del gioco: il Drone e il sistema di gestione delle sue risorse. Avendo il compito di modellare l'entità principale dell'applicazione, ho agito spesso come ponte tecnico tra il lavoro degli altri membri. Ho dovuto coordinarmi e confrontarmi spesso sia con Cristian (che si occupava della griglia e del terreno) per definire le logiche di interazione con le colture, sia con Nicholas (che si occupa del core engine e degli input) per garantire che i comandi e il movimento del drone si integrassero in modo fluido e sicuro all'interno del Game Loop generale.

Punti di forza Ritengo che il principale punto di forza del mio lavoro sia la pulizia e l'estendibilità dell'architettura che ho progettato. Aver evitato una "God Class" suddividendo le responsabilità in piccoli moduli (come `Battery`, `WaterTank` e `Inventory`) rende il codice altamente manutenibile. Anche l'implementazione del movimento fluido, basato sull'aggiornamento delta-time slegato dagli FPS, si è rivelata una buona scelta, garantendo un'esperienza visiva buona indipendente dall'hardware su cui gira il gioco.

Punti di debolezza Per quanto riguarda i punti di debolezza, riconosco che la mia implementazione lato View (`DroneView`) è rimasta piuttosto basilare. Inoltre, il sistema di rilevamento delle collisioni è solido sui limiti della mappa, ma non è stato progettato per supportare futuri ostacoli complessi interni alla griglia (come per esempio rocce o edifici).

4.1.2 Guerra Nicholas

All'interno del team di sviluppo, ho assunto principalmente il ruolo di progettista dell'architettura di sistema e di coordinatore per l'integrazione tra la logica (Model) e la grafica (View). Mi sono occupato in prima persona di sviluppare il motore dell'applicazione implementando l'architettura Game Loop e la gestione della concorrenza, in parte il sistema di cattura degli input (`InputHandler`), il cambio degli stati globali (`GameStateManager`) e l'intero sistema legato all'economia e al commercio (`Market`, `Wallet`, `PriceManager`, `Inventory`).

Punti di forza del progetto: Ritengo che il codice prodotto sia di buona qualità. L'implementazione del Game Loop con thread separati per l'aggiornamento della logica e per il rendering garantisce un'ottima scalabilità. L'uso elevato di stream, lambda e optional, unito a design pattern specifici ha reso il codice modellabile, testabile e sufficientemente modulare.

Punti di debolezza del progetto: Date le tempistiche ristrette imposte dalle scadenze, sono dovuto scendere a patti con alcuni compromessi. Ad esempio, alcune classi centrali di controllo (in particolare `InputHandler` e `App.java`) si sono espanse fino ad assumersi un po' troppe responsabilità. Una suddivisione più definita degli input (separando gli input di movimento dagli input dell'interfaccia) avrebbe sicuramente reso il codice più semplice da capire e da mantenere.

4.1.3 Qorri Cristian

All'interno del progetto, mi sono focalizzato principalmente sullo sviluppo delle meccaniche core del gioco, fungendo da ponte fondamentale tra la logica di dominio (Model), la cattura degli eventi dell'utente (Controller) e in misura mirata, la loro rappresentazione visiva (View). Il mio apporto ha riguardato la progettazione, l'implementazione e il testing di diversi sottosistemi:

- **Gestione dello stato del Drone (DroneImpl):** mi sono occupato della gestione dell'inventario interno.
- **Sistema meteorologico (WeatherManagerImpl, WeatherCondition):** ho ideato un sistema atmosferico in grado di evolversi in autonomia, innescando cambiamenti di stato sul gioco (ad esempio, pioggia che annaffia casualmente il campo e raddoppia i consumi di movimento del drone).
- **Logica di input (InputHandler):** ho gestito la cattura e lo smistamento degli eventi da tastiera, connettendo direttamente la pressione dei tasti (attraverso logiche di Key Binding e Switch Expressions) all'azione corrispondente sul campo.
- **Rendering e user interface:** mi sono occupato, per quanto riguarda la GUI, di programmare la grafica del magazzino (**StorageView**) per il trasferimento degli oggetti e gli effetti particellari a schermo necessari a rappresentare il meteo avverso (**WeatherView**).

Punti di forza del progetto

Il codice da me prodotto riflette una profonda assimilazione dei pattern Object-Oriented e delle funzionalità moderne dell'ecosistema Java:

- **Coerenza architetturale MVC:** Nello sviluppo dei miei sottosistemi ho adottato il pattern Model-View-Controller. La logica matematica e le probabilità della pioggia (**WeatherManagerImpl**) ignorano completamente l'esistenza del motore grafico; ugualmente, la **WeatherView** si limita unicamente a prelevare lo stato e a renderizzare graficamente le particelle. Questo disaccoppiamento garantisce una buona scalabilità.

Punti di debolezza del progetto

Analizzando a posteriori il lavoro svolto, riconosco alcune aree in cui le scelte architetturali o di design avrebbero potuto essere migliori, tra cui i *magic numbers*: diversi parametri, come la probabilità di pioggia (30%), il costo energetico del movimento (1.0), i modificatori malus e i timer di crescita delle piante (**Corn**, **Wheat**), risiedono nel codice sotto forma di costanti private. Questo approccio è rigido: ogni test di bilanciamento del gioco impone al programmatore di editare il file sorgente e ricompilare il progetto.

Lavori futuri e sviluppi di gruppo

Al termine dei lavori, consideriamo il risultato ottenuto un'ottima architettura di partenza. L'intento è quello di perfezionare ulteriormente il progetto per integrarlo nei nostri portfolio, rendendolo una dimostrazione concreta delle nostre capacità di progettazione e sviluppo OOP.

Per trasformare questa base in un gioco completo, gli interventi si concentreranno sulle seguenti aree:

Refactoring e Architettura software

- **Ristrutturazione del controller:** Suddivideremo le classi più corpose per rispettare il principio di Singola Responsabilità (SRP), separando nettamente la logica di I/O da quella di gioco.
- **Design pattern:** Riscriveremo il sistema di input utilizzando il **Command Pattern**, trasformando le azioni del drone ("Sposta su", "Ara", "Raccogli") in oggetti per slegarle dagli eventi della tastiera.

Grafica e Audio

- I beep di sistema (AWT) verranno sostituiti da una gestione strutturata del sound design e degli effetti audio.
- Verrà aggiunta una colonna sonora dedicata per ogni schermata.
- Sviluppo di un sistema che carichi e riproduca l'audio su thread separati, per evitare che la musica o gli effetti rallentino l'interfaccia grafica principale

Intelligenza artificiale e gameplay

- **Pathfinding:** L'introduzione di algoritmi di ricerca del percorso (come Dijkstra) permetterà al drone di eseguire ordini complessi in totale autonomia (es. "torna alla base con il percorso più breve" o "annaffia tutte le piante ignorando gli ostacoli").
- **Economia e Ambiente:** Il mercato diventerà un sistema dinamico, con prezzi influenzati dalle stagioni e da eventi meteorologici casuali. L'aggiunta del vento e dell'invecchiamento stagionale ciclico aumenterà esponenzialmente la profondità strategica del gioco.

4.2 Difficoltà incontrate e commenti per i docenti

Le sfide tecniche del progetto

Durante lo sviluppo, il problema principale non è derivato dalla logica applicativa in sé, ma dall'architettura software e dalla gestione dell'interfaccia grafica in un ambiente multithreading.

- **Game Loop e sincronizzazione:** La sfida principale è stata far convivere un Game Loop asincrono (che aggiorna costantemente la fisica su un thread separato) con Swing. Poiché Swing non è nativamente thread-safe ed è orientato agli eventi piuttosto che ai tick continui, i tentativi di lettura simultanea per il rendering causavano race conditions, glitch visivi e eccezioni. Abbiamo risolto il problema approfondendo la mutua esclusione, applicando correttamente i blocchi `synchronized` e i `Lock Objects` al Model senza causare deadlock nella UI.
- **Gestione degli input avanzata:** L'utilizzo dei classici `KeyListener` per il controllo del drone si è rivelato inaffidabile a causa delle continue perdite di focus passando tra i menu. Cercando online e su intelligenza artificiale, abbiamo implementato i `Key Bindings`, garantendo controlli fluidi e reattivi.
- **Rendering custom (Graphics2D):** La realizzazione del sistema meteorologico ha richiesto l'override del metodo `paintComponent()`. È stato necessario un attento lavoro di ottimizzazione per gestire il rendering simultaneo di oltre 100 particelle senza far crollare gli FPS e lasciare il gioco fluido.

Feedback e suggerimenti per il corso

Il corso fornisce basi teoriche solide sui fondamenti dell'Object-Orientation e sui pattern architetturali (come l'MVC) e il progetto finale è un buon metodo per farci avvicinare all'ottica lavorativa di gruppo. Un suggerimento che ci viene da proporre riguarda il seguente tema:

- **Interazione Thread/GUI e Game Loop:** Sarebbe estremamente utile un piccolo modulo didattico dedicato a come interfacciare i pattern classici con un motore a tempo continuo. Come quello che ha tenuto il professor Ricci nel 2024. Fornire best-practice su come sincronizzare in sicurezza un thread in background con la UI eviterebbe a molti gruppi di sprecare ore senza risolvere nulla (come noi).

A Guida utente

1. Introduzione

Benvenuto in **Agro-Bot**, un videogioco gestionale con visuale 2D top-down. In questo titolo controllerai un drone agricolo di ultima generazione con l'obiettivo di coltivare, raccogliere e vendere prodotti agricoli per massimizzare il tuo profitto.

Il tuo compito principale sarà quello di bilanciare le attività agricole (arare, seminare, raccogliere) con l'efficienza logistica: ogni azione consumerà infatti la batteria del tuo drone e l'acqua del tuo serbatoio, costringendoti a gestire con attenzione le tue risorse.

2. Controlli di base

Per interagire con il mondo di Agro-Bot, utilizza i seguenti tasti della tastiera.

2.1 Movimento

- **W** oppure **Freccia su**: Sposta il drone in alto.
- **S** oppure **Freccia giù**: Sposta il drone in basso.
- **A** oppure **Freccia sinistra**: Sposta il drone a sinistra.
- **D** oppure **Freccia destra**: Sposta il drone a destra.

2.2 Azioni agricole

- **C** (Plow): Ara il terreno. Prepara una normale casella di terra rendendola adatta ad accogliere un seme.
- **I** (Irrigate): Annaffia il terreno. Questa azione consuma anche l'acqua del serbatoio del drone.
- **P** (Plant): Pianta un seme sulla casella in cui ti trovi. (Nota: il terreno deve essere stato precedentemente arato e devi avere un seme selezionato nel tuo inventario).
- **H** (Harvest): Raccogli una pianta che ha completato il suo ciclo di crescita ed è giunta a maturazione.
- **U** (Use herbicide): Usa il diserbante per rimuovere le fastidiose erbacce che possono colpire le zolle non arate.

2.3 Gestione dell'inventario

Tasti 1, 2, 3: Seleziona lo slot corrispondente nel tuo inventario rapido. Utilizza questi tasti per scegliere attivamente quale seme piantare.

2.4 Interazioni con gli Edifici Speciali

Per interagire con gli edifici, posiziona il drone sopra la casella della struttura e usa i seguenti tasti:

- **M** (Menu/Market):
 - Sull'**Hangar**: Apre il menu del Magazzino per gestire le tue scorte.
 - Sul **Mercato**: Apre il negozio dove puoi usare i tuoi crediti per comprare semi (es. Grano, Mais), diserbanti e vendere i raccolti per fare profitto.
- **R** (Recharge/Refill):
 - Sull'**Hangar**: Ricarica completamente la batteria del drone.
 - Sul **Pozzo**: Ricarica il serbatoio dell'acqua a bordo.

2.5 Menu di Gioco

ESC: Mette in pausa il gioco o chiude qualsiasi menu attualmente aperto per ritornare alla partita.

2.6 Consigli

Potrai sempre rivedere le funzioni dei vari tasti attraverso gli appositi pulsanti per i comandi (i) e guida utente (?).

3. Primi passi: Il ciclo di gioco

Se è la prima volta che giochi, ecco una breve guida per iniziare a guadagnare i tuoi primi crediti:

1. **Preparazione:** Inizii la partita posizionato sopra l'Hangar. Controlla il tuo inventario per verificare di avere dei semi. In caso contrario, dirigiti verso la casella del mercato, premi M e acquista i tuoi primi semi.
2. **Lavorazione:** Dirigiti su una casella di terra libera. Premi C per arare la terra. Se noti la comparsa di erbacce, premi U per rimuoverla, in modo da poterla arare.
3. **Semina:** Scegli il seme dal tuo inventario usando i tasti numerici (1, 2 o 3), poi premi P per piantarlo nel terreno arato.
4. **Cura della pianta:** Premi I per irrigare la pianta.
5. **Raccolto e profitto:** Quando la pianta raggiunge lo stadio finale, premi H per raccoglierla. Vai subito al mercato e premi M per venderla.
6. **Sopravvivenza:** Tieni sempre d'occhio l'interfaccia (HUD): non lasciare che la tua batteria si scarichi a zero (torna all'Hangar e premi R per ricaricare) o il drone si spegnerà! Fai la stessa cosa recandoti al Pozzo per riempire l'acqua quando è esaurita.